

ResolveX

AI-Assisted Grievance and Escalation Management System

Project Proposal for Software Engineering Lab (Batch: 3C34)

Nikhil Garg* Bhumi Garg† Sidhhay Abrol‡

Thapar Institute of Engineering and Technology

August 18, 2026

1 Higher-order goal . . . institutional accountability

ResolveX aims to improve institutional governance in colleges and organizations by replacing informal, easily-forgotten complaint handling with a structured, accountable digital workflow. While the underlying problem is broader than any single institution, the immediate engineering objective is to translate transparent grievance handling into a measurable, deployable software system that gives every complaint a clear owner and a bounded resolution timeline.

2 Time-to-value . . . primary heuristic

- We will deliver meaningful increments quickly by building the complaint lifecycle and role-based dashboards first, then layering AI classification and escalation on top of a working core.
- We will prioritize fast feedback over perfection by validating authentication, complaint CRUD, and status tracking in the early weeks before investing in AI and analytics.
- Success is defined by what can be delivered and demonstrated at each milestone (Weeks 1–15), then improved based on user and stakeholder feedback.

3 Problem Statement

In most colleges and organizations, complaints submitted by students or employees are handled informally, if at all. Common pain points include:

- **No formal record:** complaints are frequently ignored or lost with no trace.
- **Low transparency:** complainants cannot see the status of their issue.
- **No escalation mechanism:** when a responsible department fails to act, there is no automatic path to a higher authority.

*Roll No: 1024030504, GitHub: github.com/NikhilGarg-1

†Roll No: 1024031096, GitHub: github.com/Bhumigarg1605

‡Roll No: 1024031157, GitHub: github.com/sidhhayResolveX

- **No accountability:** departments have no measurable performance record.

Why this matters now (contextual relevance): A student reporting non-functional hostel Wi-Fi today has no way to ensure the complaint is even seen by the right person, let alone resolved within a reasonable time. Even a lightweight, automatically-enforced workflow can meaningfully reduce complaints that are silently dropped.

4 Proposed Solution ... Software Proposition

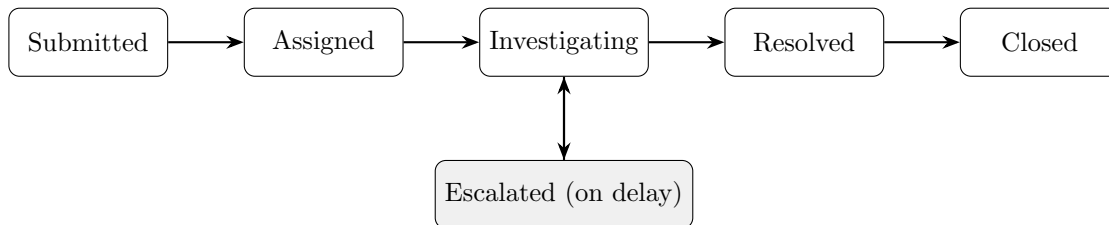
4.1 Overview

Build a web-based grievance and escalation management platform with the following components:

- **Citizen (complainant) portal:** students/employees submit complaints with optional images or documents.
- **AI-based triage:** automatic classification of each complaint into the correct category and department, with a rule-based fallback.
- **Department workflow:** role-based dashboards for handlers to update status and add comments.
- **Escalation engine:** complaints that miss their SLA window are automatically passed up the chain of authority.
- **Admin/ops dashboard:** departmental performance analytics and backlog visibility.

4.2 Core workflow: the complaint lifecycle

The complaint lifecycle is the core of ResolveX's design and maps naturally onto a *State* design pattern, since the set of actions allowed on a complaint changes with its status.



Escalation rule (example SLA):

- No action within 3 days of assignment → escalate to Head of Department.
- No further action within 5 days → escalate to Dean / Director.

4.3 Operational constraints for an educational setting

- Role-based dashboards (student, handler, admin) accessible on low-to-mid range devices.
- AI classification is a strategy, not a hard dependency: a rule-based/keyword fallback keeps the system usable if the LLM service is unavailable.
- Design for deployability using common managed hosting rather than custom infrastructure.

5 Solution Approach ... Engineering focus

This is an engineering course project, so we focus on demonstrable software design and workflow correctness rather than novel AI research.

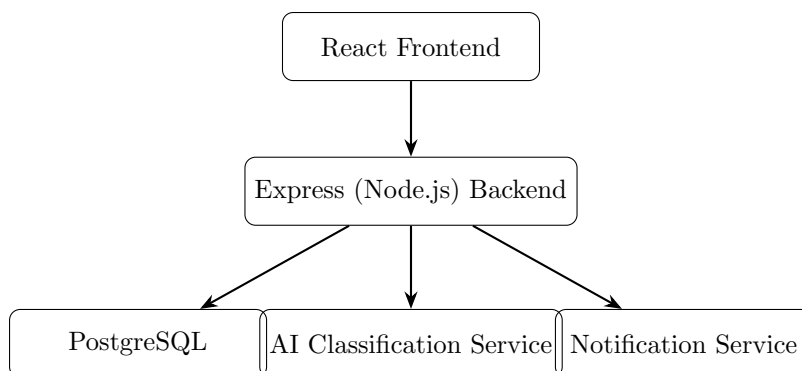
5.1 Design patterns ... non-research

Four design patterns map cleanly onto the problem, giving the project a concrete, demonstrable software-design backbone rather than an ad-hoc implementation.

Pattern	Where it is used
State	Complaint behaviour changes with its status (Submitted, Investigating, Resolved, Closed).
Chain of Responsibility	Escalation passes the complaint up the chain: Handler → HOD → Dean → Director.
Observer	Notifies the student, handler, and admin whenever a complaint's status changes.
Strategy	AI classification can switch between keyword matching, a trained classifier, or an LLM.

5.2 Web architecture ... web-based solution

A typical decoupled web architecture:



- **Frontend:** React.js + Tailwind CSS, three role-based dashboards (student, handler, admin).
- **Backend API:** Node.js + Express, REST endpoints for authentication, complaint management, and escalation.
- **Database:** PostgreSQL — complaints, users, departments, and escalations are naturally linked by foreign keys.
- **Auth:** JWT for signed session tokens, bcrypt for password hashing, role-checking middleware on every protected route.

5.3 Notifications and real-time updates

Socket.IO pushes real-time in-app notifications on every status change; Nodemailer sends corresponding email notifications. This gives the Observer pattern a concrete transport layer rather than relying on polling.

6 Evaluation Criterion ... measurable and attributable

We define success using metrics tied to the core complaint lifecycle.

6.1 Primary evaluation metric

Time-to-Assignment (TTA): time between complaint submission and the first *Assigned* status update in the system.

- Target: median TTA ≤ 2 hours during pilot window.
- Attribution: measured directly from database timestamps (`created_at` vs. first status transition).

6.2 Secondary evaluation metrics

- **SLA compliance rate:** percentage of complaints resolved before triggering an escalation.
- **Escalation accuracy:** escalated complaints that were, in fact, delayed beyond the SLA window (no false escalations).
- **Classification accuracy:** percentage of complaints correctly auto-categorized by the AI service versus staff-corrected category.
- **User clarity:** complainant-reported clarity of status using a simple 1–5 feedback question after resolution.

6.3 Pilot validation plan ... high-level

- Recruit 10–20 student complainants and 3–5 department handlers (simulated departments if real staff are unavailable).
- Compare TTA and SLA compliance before/after by logging baseline estimates from informal process interviews.

7 Scalability ... with foresight

The system should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** supports growth in complaint volume and concurrent dashboard usage by
 - decoupling frontend and backend,
 - indexing common queries (status, department, `created_at`),
 - using stateless, token-based API design.
2. **Operationally deployable (practically):** runs on common managed hosting — Vercel (frontend), Render/Railway (backend), and Supabase (PostgreSQL) — with Cloudinary for attachment storage.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project, so effort stays focused on workflow design and correctness rather than infrastructure.

Layer	Technology
Frontend	React.js + Tailwind CSS
Backend	Node.js + Express.js
Database	PostgreSQL
Authentication	JWT (JSON Web Tokens) + bcrypt for password hashing
AI Service	LLM API for complaint classification, with a rule-based fallback
Notifications	Socket.IO (real-time) + Nodemailer (email)
File Storage	Cloudinary (images / documents attached to complaints)
Deployment	Vercel (frontend) + Render/Railway (backend) + Supabase (PostgreSQL)

9 Project scope and deliverables ... milestone-friendly

9.1 Database design

Table	Key Fields
Users	user_id, name, email, password_hash, role, department_id
Complaints	complaint_id, title, description, category, status, created_by, assigned_to, created_at
Departments	department_id, name, handler_id
Escalations	escalation_id, complaint_id, level, reason, timestamp
Notifications	notification_id, user_id, message, read_status

9.2 Timeline (15 weeks)

Weeks	Milestone
1–2	Requirements gathering, use-case & UML diagrams
3–4	Database design and schema setup
5–7	Authentication (JWT + bcrypt) and core complaint CRUD APIs
8–9	AI classification service integration
10–11	Escalation engine (Chain of Responsibility) and notifications (Observer)
12–13	Frontend dashboards for all three roles
14	Testing (unit, integration, and user acceptance)
15	Deployment, documentation, and final presentation

9.3 Expected outcomes

- A working web application with role-based dashboards for students, handlers, and admins.

- Automatic complaint classification using an AI service.
- A working escalation engine that reassigns complaints past their SLA.
- Real-time and email notifications on every status change.
- Complete UML documentation: use-case, class, state, and ER diagrams.

10 Risks and mitigations

- **AI misclassification:** keep a rule-based/keyword fallback (Strategy pattern) and let handlers manually recategorize; never auto-close on a low-confidence classification.
- **Staff/handler adoption:** keep the status-update UI minimal, with one-click transitions and pre-filled comment templates.
- **False or excessive escalations:** log every escalation trigger with its SLA timestamp so thresholds can be tuned before the pilot.
- **Data privacy:** minimize collected data, keep attachments optional, and restrict access by role via JWT-checked middleware.
- **Operational issues in pilot:** use a staging environment early; keep CI/CD checks (build, lint, tests) on every change to ensure repeatable deployments.

11 Summary

ResolveX proposes a deployable, AI-assisted grievance and escalation platform that replaces informal complaint handling in colleges and organizations with a structured, accountable workflow. It meets the course criteria by grounding the system in four well-motivated design patterns (State, Chain of Responsibility, Observer, Strategy), a relational database with meaningful constraints, measurable evaluation criteria (particularly Time-to-Assignment and SLA compliance), and a realistic 15-week implementation plan achievable for a three-member team.